

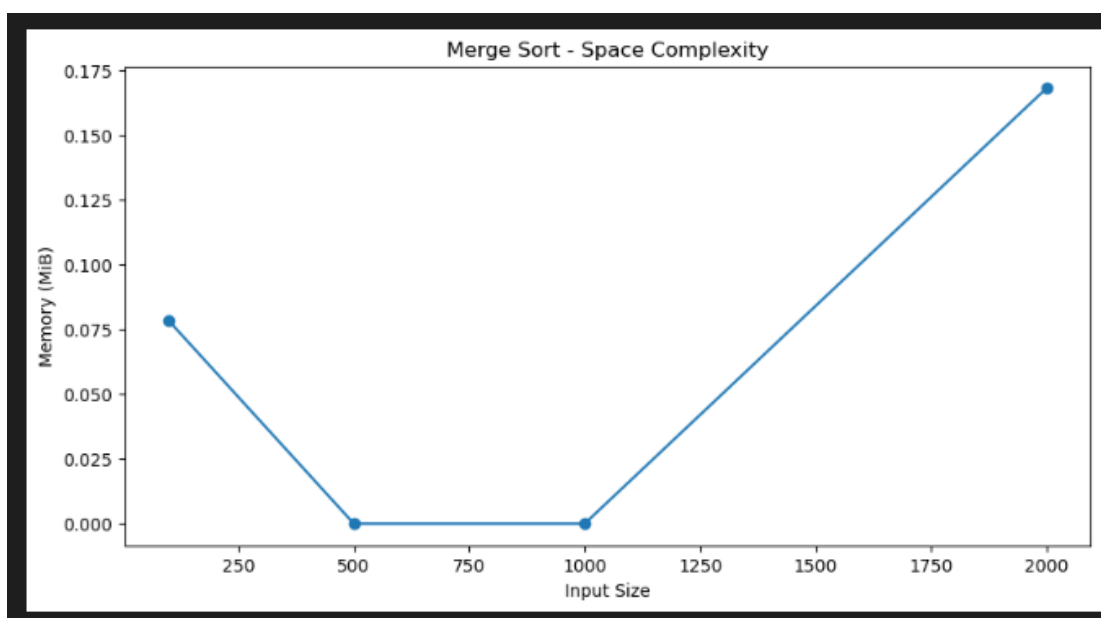
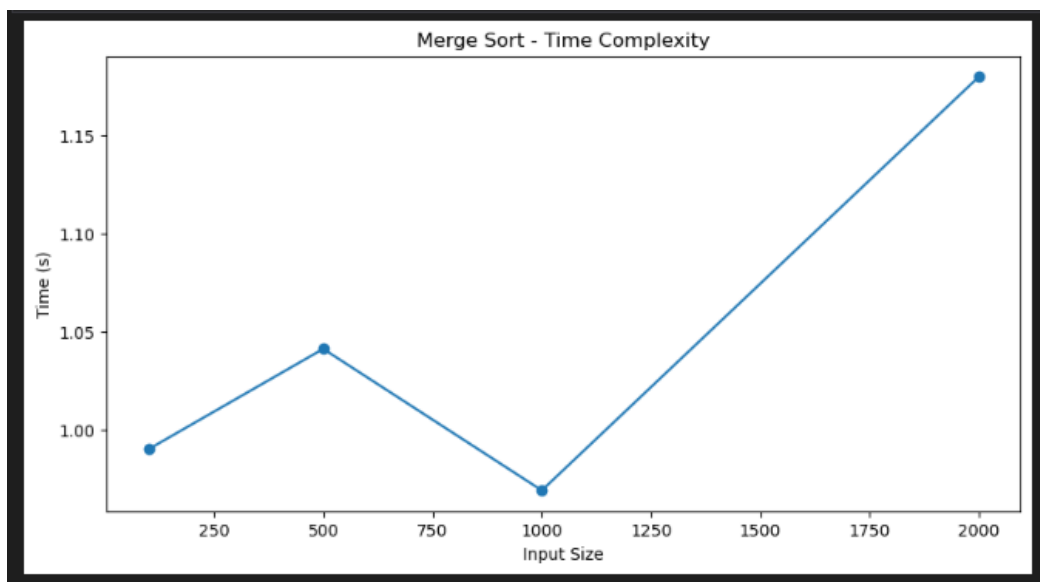
Lab Assignment – 1

1. Merge Sort

Time Complexity: $O(n \log n)$ (best, average, worst)

Space Complexity: $O(n)$

Stable and reliable but uses more memory.

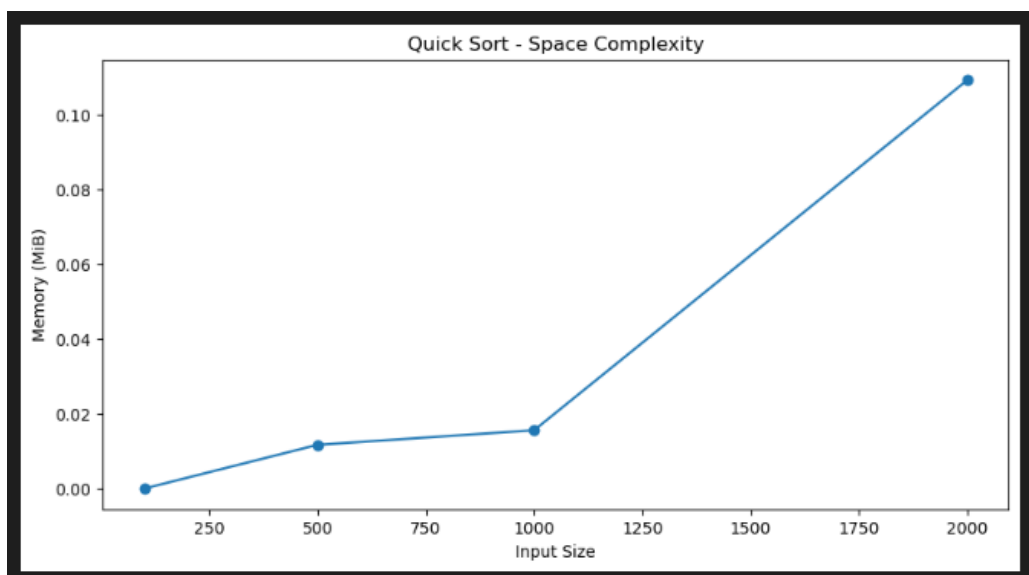
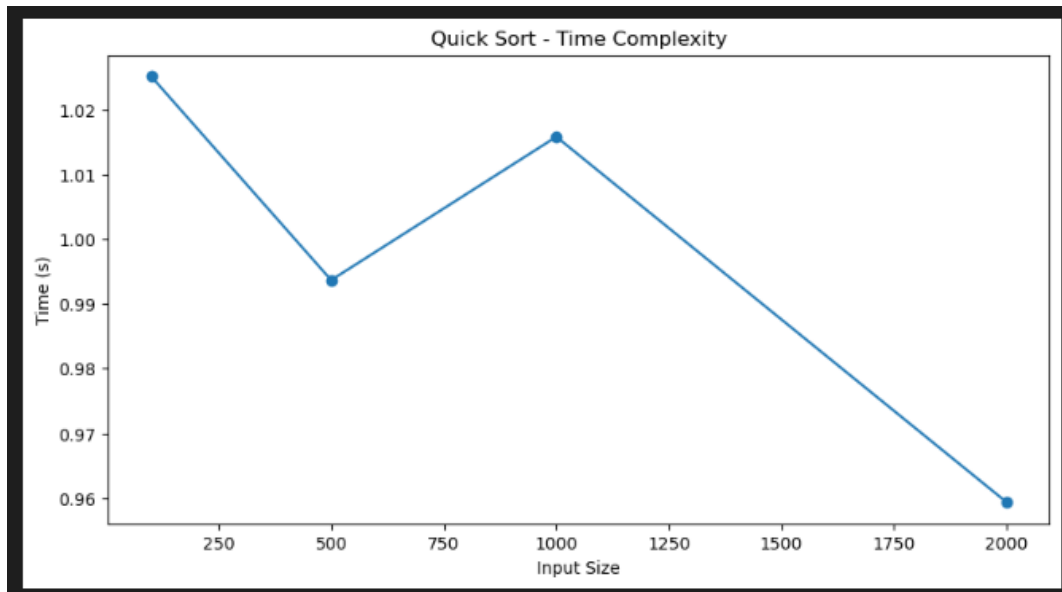


2. Quick Sort

Time Complexity: Best & Average $O(n \log n)$, Worst $O(n^2)$

Space Complexity: $O(\log n)$

Very fast in practice but recursive, so risk of stack overflow.

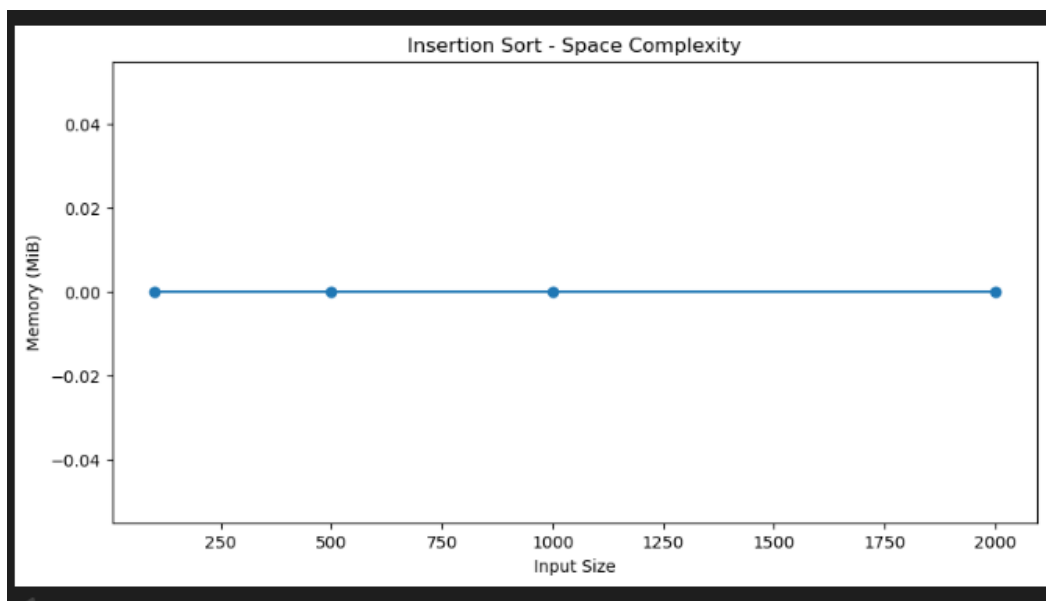
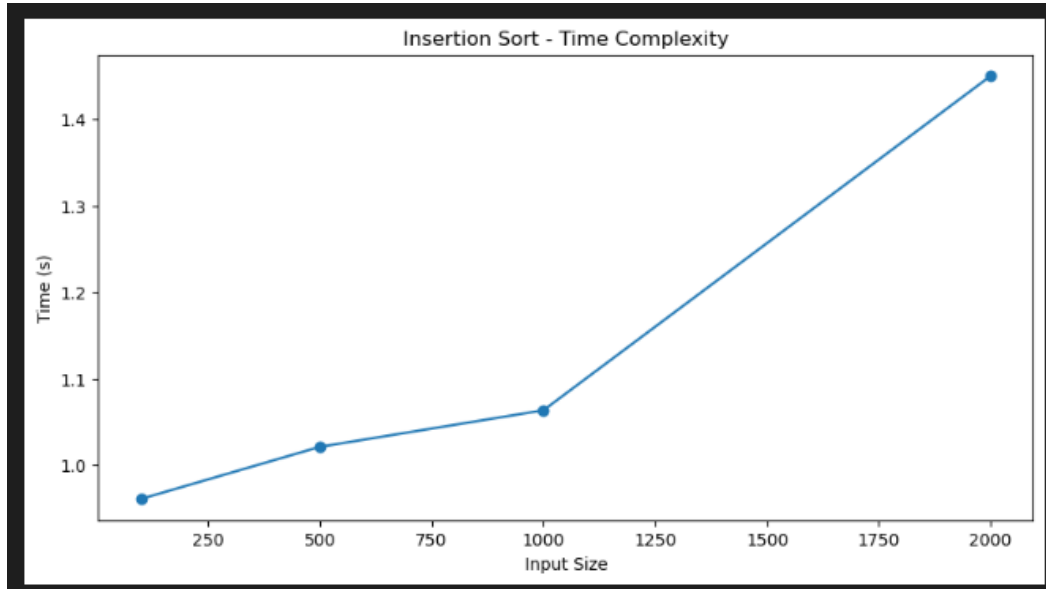


3. Insertion Sort

Time Complexity: Best $O(n)$, Worst $O(n^2)$

Space Complexity: $O(1)$

Works well for small or nearly sorted data.

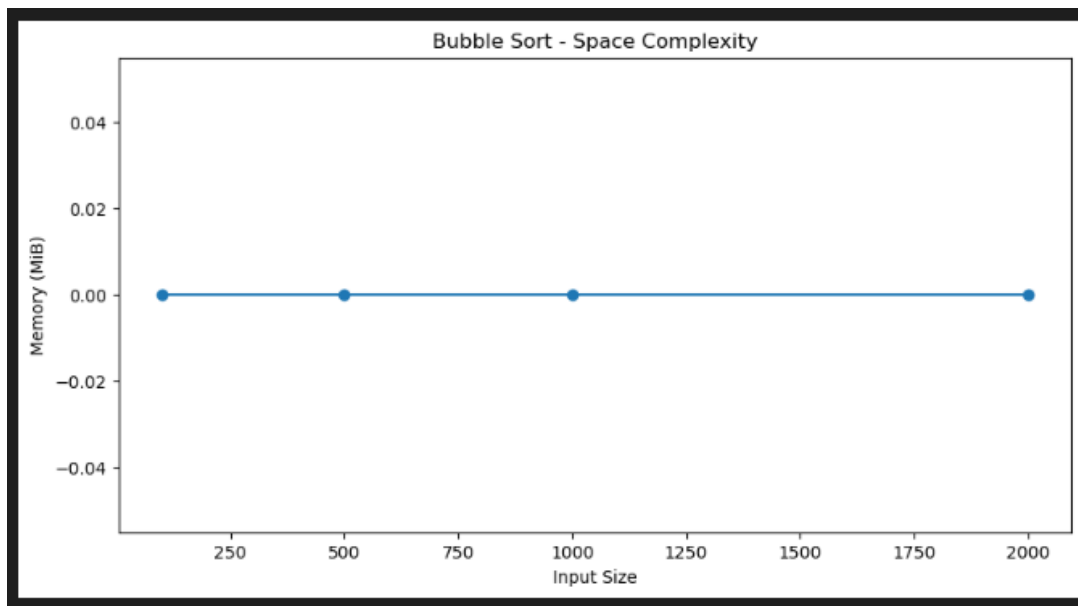
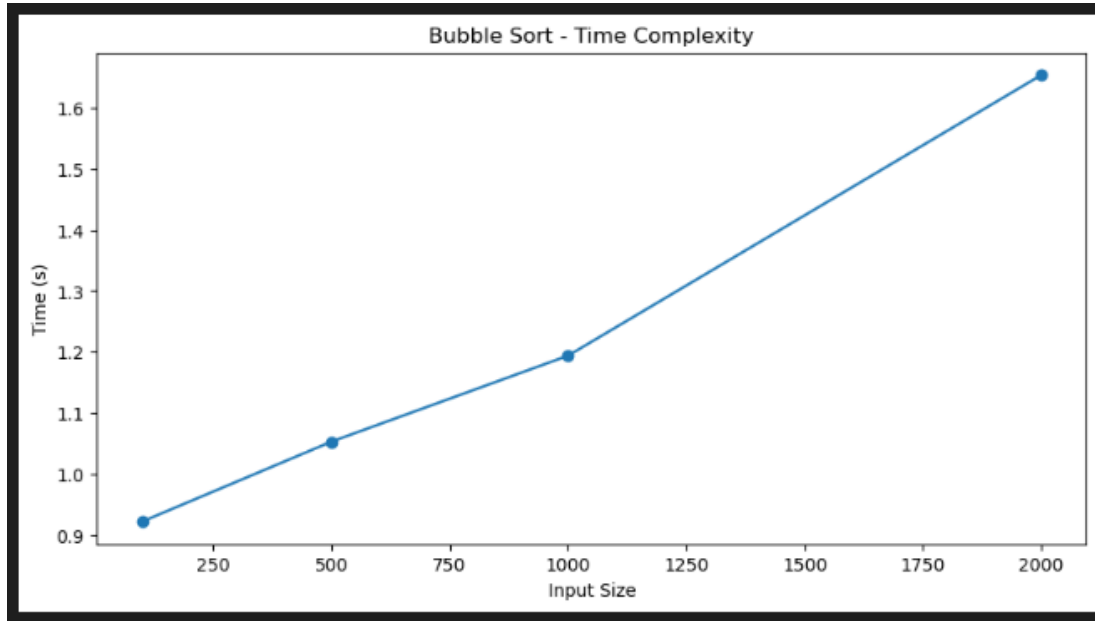


4. Bubble Sort

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Simple but very slow.

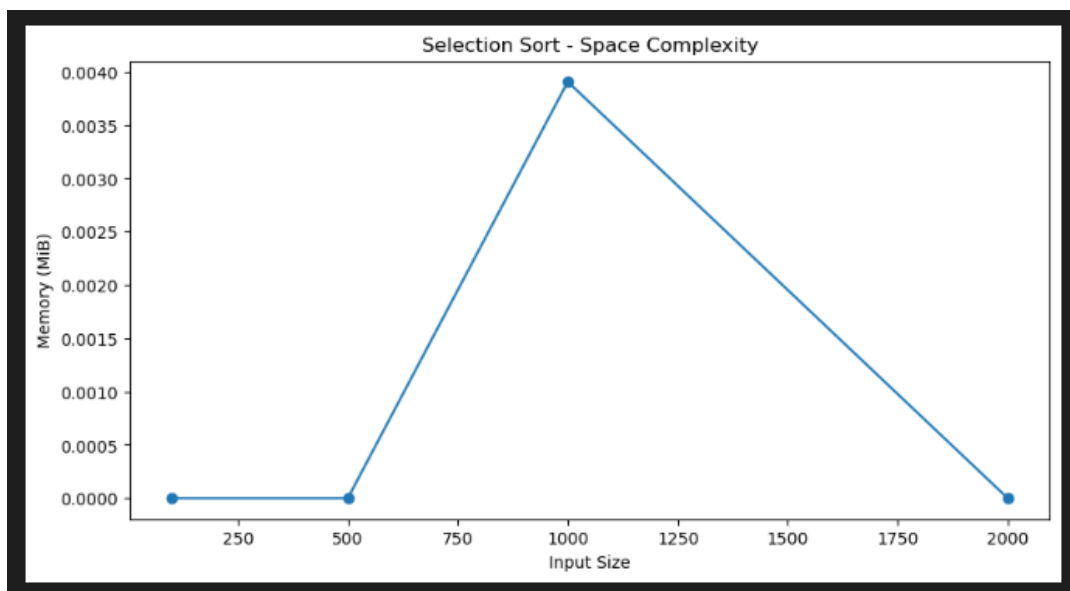
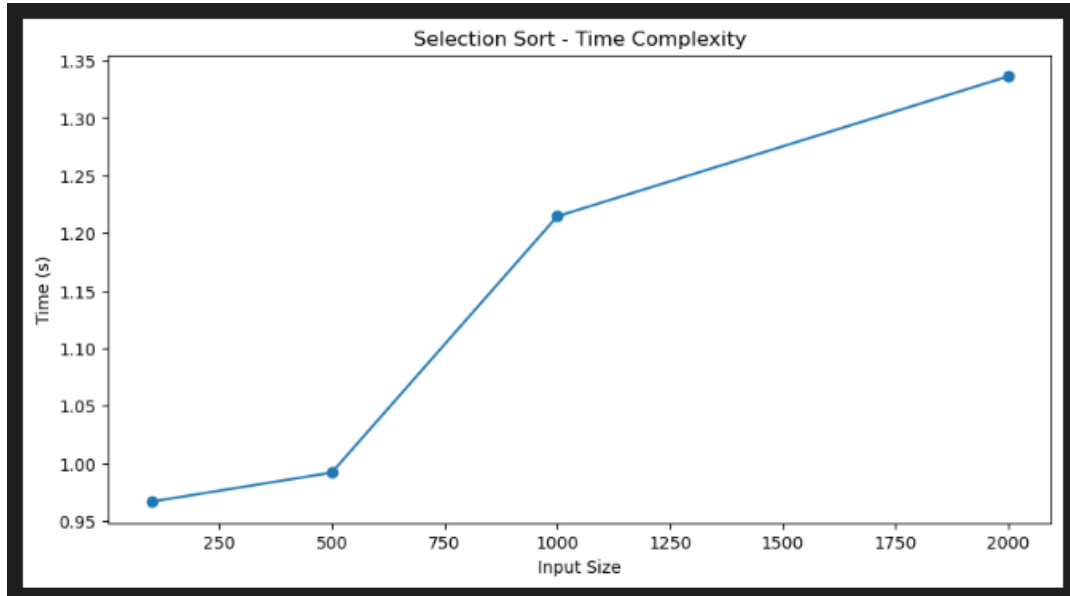


5. Selection Sort

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Simple, not practical for large datasets.



6. Fibonacci (Dynamic Programming)

Input: A number n

Output: n th Fibonacci number

Time Complexity: $O(n)$

Space Complexity: $O(n)$ (can be optimized to $O(1)$)

Much faster and scalable than recursive.

7. Fibonacci (Recursive)

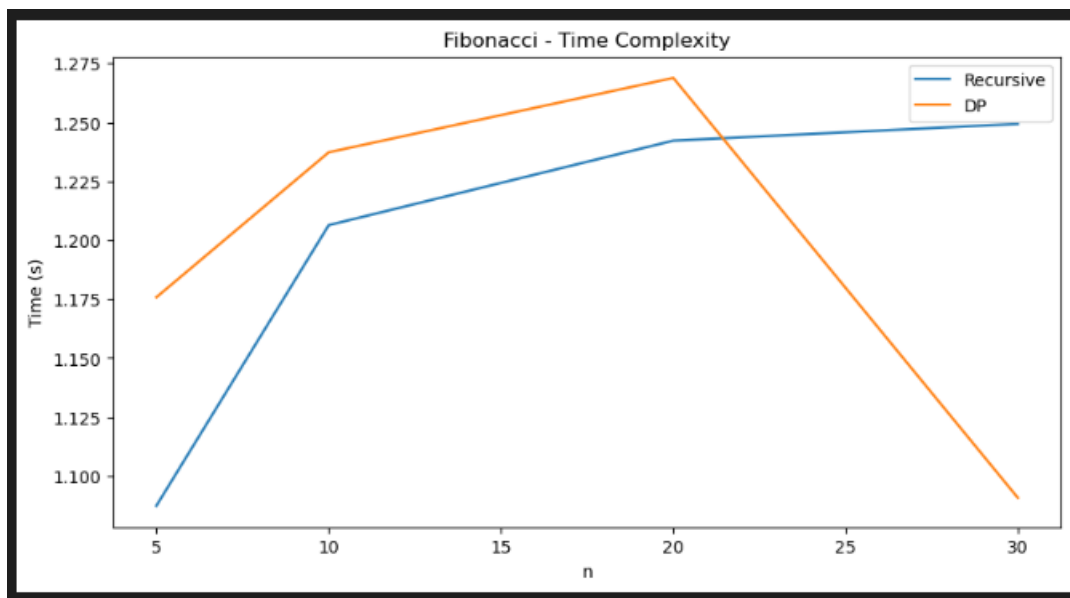
Input: A number n

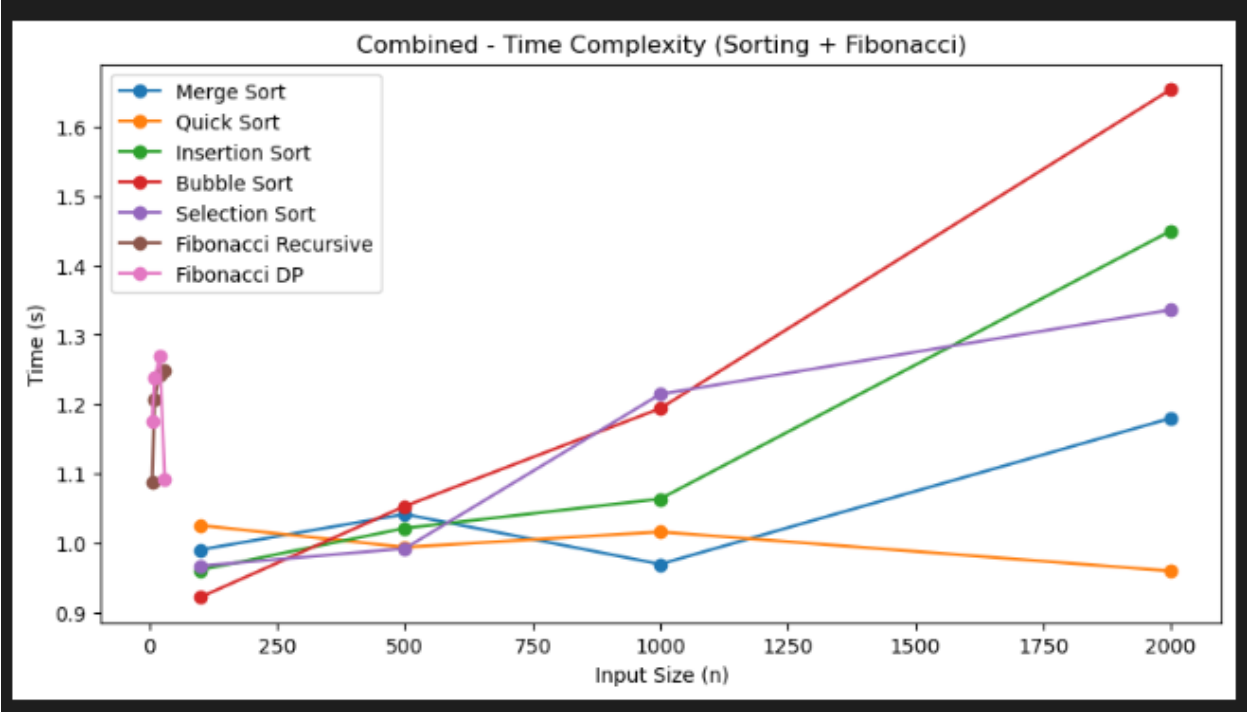
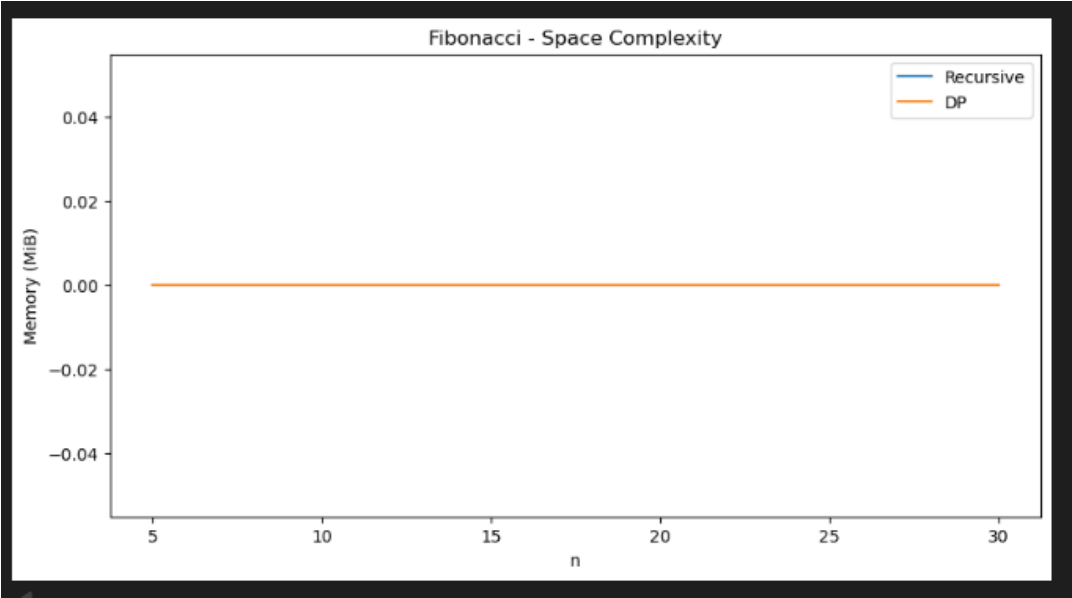
Output: n th Fibonacci number

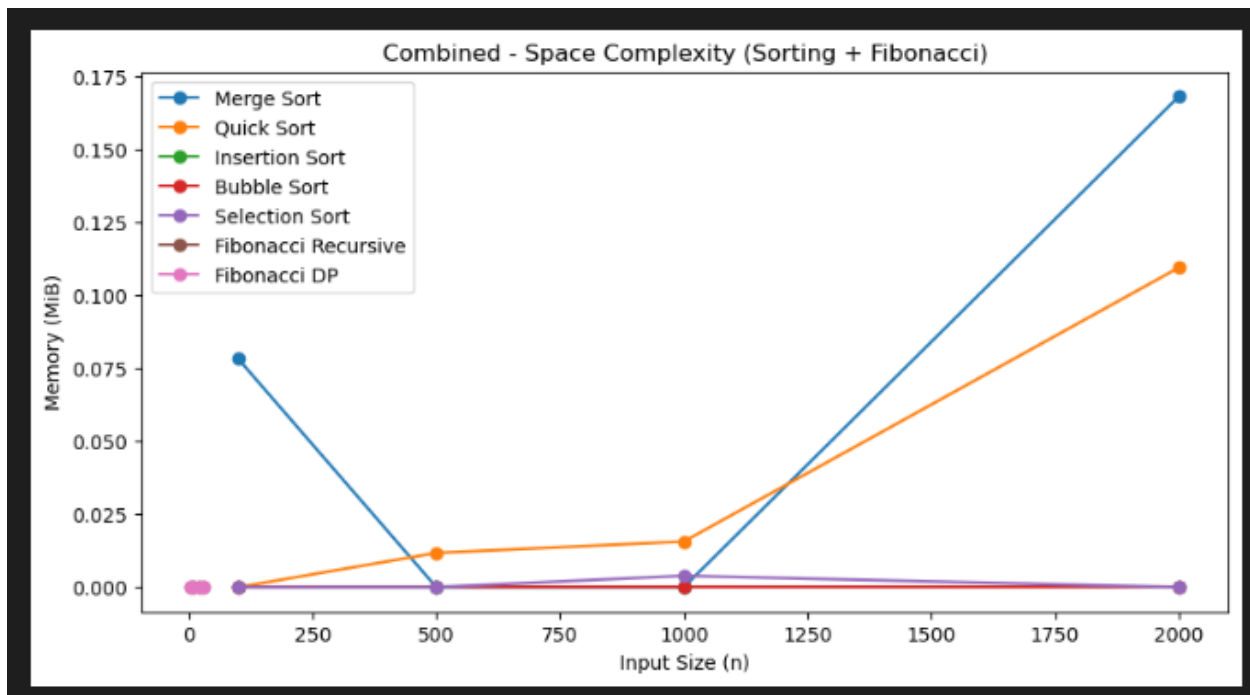
Time Complexity: Exponential $O(2^n)$

Space Complexity: $O(n)$ (recursion stack)

Suitable only for very small inputs. Risk of stack overflow if n is large.







1. Execution Time vs. Input Size using matplotlib

The plot shows that execution time rises with input size for all algorithms.

Efficient ones (Merge Sort, Quick Sort, Fibonacci DP) handle growth well, while slower ones (Bubble Sort, Selection Sort, Recursive Fibonacci) scale poorly.

The graphs highlight clear contrasts between $O(n \log n)$, $O(n^2)$, and $O(2^n)$ complexities.

2. Time–Space Trade-offs

Merge Sort uses extra memory ($O(n)$) for merging but ensures $O(n \log n)$ speed.

Quick Sort is usually faster and more memory-efficient but can degrade to $O(n^2)$ in the worst case.

Bubble, Insertion, and Selection Sorts need little memory ($O(1)$) but are slow ($O(n^2)$).

Recursive Fibonacci uses minimal space yet is very slow, while DP Fibonacci trades memory for $O(n)$ performance.

-- Faster algorithms often need more memory; space-saving ones usually take longer.

3. Recursion Depth & Stack Overflow

Recursive Fibonacci's depth grows with n , risking stack overflow for large inputs (Python limit ≈ 1000).

Merge and Quick Sort have safe recursion depths of $O(\log n)$, and DP Fibonacci avoids recursion entirely.

-- Deep recursion mainly threatens simple recursive methods like Fibonacci.

